

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ОТЧЕТ

о преддипломной (производственной) практике

наименование вида и типа практики

на (в) ООО «Предприятие ВТИ-Сервис»

наименование предприятия, организации, учреждения

Студента 4курса, группы ПО-22б

курса, группы

Петрова Михаила Евгеньевича

фамилия, имя, отчество

Руководитель практики от
предприятия, организации,
учреждения

Оценка _____

должность, звание, степень

фамилия и. о.

подпись, дата

Руководитель практики от
университета

Оценка _____

К.Т.Н. доцент

должность, звание, степень

Чаплыгин А. А.

фамилия и. о.

подпись, дата

Члены комиссии

подпись, дата

фамилия и. о.

подпись, дата

фамилия и. о.

подпись, дата

фамилия и. о.

Курск 2026 г.

СОДЕРЖАНИЕ

1	Анализ предметной области	6
1.1	Характеристика деятельности редакционно-издательского отдела вуза	6
1.2	Проблематика традиционного подхода к организации проверки материалов	7
1.3	Обоснование необходимости автоматизации	8
1.4	Анализ существующих систем автоматизации документооборота и проверки документов	9
1.4.1	Microsoft Word и встроенные средства рецензирования	9
1.4.2	Google Docs	9
1.4.3	Системы электронного документооборота	10
1.4.4	Вывод по анализу аналогов	10
1.5	Постановка задач на разработку	11
2	Техническое задание	12
2.1	Основание для разработки	12
2.2	Цель и назначение разработки	12
2.3	Требования к функциональности	12
2.4	Требования к интерфейсу	14
2.5	Требования к обработке ошибок	17
2.6	Требования к удобству использования (UX)	18
2.7	Моделирование вариантов использования	18
2.7.1	Вариант использования «Вход в систему»	21
2.7.2	Вариант использования «Выход из системы»	22
2.7.3	Вариант использования «Загрузка документа на проверку»	22
2.7.4	Вариант использования «Автоматическая проверка документа»	23
2.7.5	Вариант использования «Получение отчёта об ошибках»	23
2.7.6	Вариант использования «Скачивание документа с выделенными ошибками»	24

2.7.7	Вариант использования «Автоматическое сохранение корректного документа»	24
2.7.8	Вариант использования «Просмотр архива документов»	25
2.7.9	Вариант использования «Формирование титульного листа»	25
2.7.10	Вариант использования «Просмотр статистики»	26
2.7.11	Вариант использования «Управление пользователями»	26
2.7.12	Вариант использования «Настройка параметров системы»	26
2.8	Требования к надёжности	27
2.9	Требования к информационной безопасности	28
2.10	Требования к программной и технической совместимости	28
2.11	Требования к оформлению документации	29
3	Технический проект	30
3.1	Архитектура программной системы	30
3.2	Модуль обработки учебно-методических документов	32
3.2.1	Общая архитектура модуля	32
3.2.2	Интеграция с Microsoft Word через COM-интерфейс	32
3.2.3	Алгоритм автоматической проверки оформления	33
3.2.4	Формирование исправленной копии документа	37
3.2.5	Генерация PDF-отчёта	37
3.2.6	Генерация титульных листов	38
3.2.7	Управление сессиями и обновление токенов	39
3.2.8	Взаимодействие с базой данных через ORM	40
3.2.9	Предотвращение дублирования	41
3.2.10	Обработка ошибок и восстановление	42
3.2.11	Хранение данных	42
3.2.12	Логирование	43
3.2.13	Заключение	43
3.3	База данных и её проектирование	44
3.3.1	ER-модель данных	44
3.3.2	Нормализация	46
3.3.3	Реляционная модель данных	46

3.3.4 Описание структуры базы данных	48
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	50

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

РП — рабочий проект;

ТЗ — техническое задание;

ТП — технический проект;

JPEG (Joint Photographic Experts Group) — растровый графический формат изображений и фотографий с высокой степенью сжатия;

DCT (Discrete Cosine Transform) — дискретное косинусное преобразование, используемое в алгоритме сжатия JPEG;

RGB (Red, Green, Blue) — цветовая модель, основанная на представлении цвета с использованием красного, зелёного и синего компонентов;

YCbCr — цветовое пространство, включающее яркостную компоненту (Y) и две цветоразностные компоненты (Cb, Cr);

GUI (Graphical User Interface) — графический пользовательский интерфейс;

API (Application Programming Interface) — программный интерфейс взаимодействия компонентов системы;

IDCT (Inverse Discrete Cosine Transform) — обратное дискретное косинусное преобразование;

ROI (Region of Interest) — область интереса изображения, используемая при анализе и обработке данных.

1 Анализ предметной области

1.1 Характеристика деятельности редакционно-издательского отдела вуза

Редакционно-издательский отдел (РИО) высшего учебного заведения является структурным подразделением, обеспечивающим подготовку, проверку, оформление и выпуск учебно-методической, научной и справочной литературы, используемой в образовательном процессе. К числу основных материалов, поступающих в РИО, относятся методические указания, учебные пособия, практикумы, монографии, сборники научных трудов и иные издания.

Современный образовательный процесс требует постоянного обновления учебно-методической базы. Изменение федеральных государственных образовательных стандартов, развитие научных направлений, внедрение новых технологий обучения и цифровых платформ обуславливают регулярную подготовку новых материалов преподавателями университета.

РИО выполняет функции организационного и нормативного сопровождения издательской деятельности вуза. Сотрудники подразделения принимают материалы от авторов, проверяют их соответствие установленным требованиям оформления, контролируют наличие обязательных структурных элементов, подготавливают замечания, формируют титульные листы, а также ведут учёт поступивших и утверждённых работ.

В условиях большого количества авторов и поступающих документов особую значимость приобретают скорость обработки материалов, точность проверки и удобство взаимодействия между преподавателями и сотрудниками отдела. Эффективность деятельности РИО напрямую влияет на своевременное обеспечение образовательного процесса актуальными учебными материалами.

1.2 Проблематика традиционного подхода к организации проверки материалов

Традиционный подход к работе редакционно-издательского отдела, основанный на ручной обработке документов, сопровождается рядом существенных недостатков, снижающих эффективность работы подразделения:

1. Высокая трудоёмкость проверки оформления. В большинстве случаев сотрудники РИО вручную проверяют поступившие документы на соответствие установленным требованиям: параметры шрифта, интервалы, отступы, структура заголовков, оформление списков литературы, таблиц, рисунков и других элементов. При большом количестве работ такой процесс занимает значительное время.

2. Высокая вероятность человеческих ошибок. Ручная проверка не гарантирует обнаружение всех нарушений оформления. Из-за большого объёма текста, однотипности операций и человеческого фактора часть ошибок может остаться незамеченной, что приводит к возврату материалов на доработку уже на поздних этапах.

3. Длительное взаимодействие с авторами. После выявления замечаний сотрудник вынужден вручную составлять перечень ошибок либо вносить исправления в режиме рецензирования документа. Это увеличивает сроки согласования и повторной подачи материалов.

4. Отсутствие централизованного хранилища работ. Готовые и поступившие материалы часто хранятся в отдельных папках, на персональных компьютерах или сетевых дисках. Это затрудняет поиск ранее утверждённых изданий, ведение архива и получение статистических сведений.

5. Сложность формирования отчётности. Для подготовки отчётов по количеству проверенных работ, числу авторов, срокам обработки и видам материалов требуется вручную собирать данные из различных источников. Это увеличивает нагрузку на сотрудников и снижает оперативность получения информации.

6. Отсутствие унифицированных шаблонов. Авторы нередко готовят титульные листы и иные элементы документа самостоятельно, что приводит к многочисленным отклонениям от утверждённых образцов и повторным исправлениям.

Таким образом, ручной подход к организации работы РИО не обеспечивает необходимой скорости, точности и удобства обработки документов в современных условиях.

1.3 Обоснование необходимости автоматизации

Решение перечисленных проблем возможно путём внедрения специализированной информационно-аналитической системы для автоматизации деятельности редакционно-издательского отдела вуза.

Разрабатываемая система должна обеспечить:

- загрузку пользователями учебно-методических и научных материалов через веб-интерфейс;
- автоматическую проверку документов на соответствие требованиям оформления;
- формирование подробного отчёта с перечнем найденных ошибок;
- автоматическое создание файла формата DOCX с выделенными нарушениями;
- автоматическое сохранение корректно оформленных работ в централизованном хранилище;
- ведение статистики поступивших, проверенных и утверждённых материалов;
- генерацию титульных листов по утверждённым шаблонам;
- разграничение прав доступа между авторами, редакторами и администраторами системы;
- удобный интерфейс взаимодействия пользователей с системой.

Автоматизация позволит существенно сократить время проверки материалов, снизить количество ошибок, ускорить процесс согласования документов и обеспечить прозрачный контроль деятельности подразделения.

1.4 Анализ существующих систем автоматизации документооборота и проверки документов

На современном рынке представлены различные программные решения, связанные с управлением документами, совместной работой и проверкой текстовых материалов. Однако большинство из них не ориентировано на специфику работы редакционно-издательского отдела вуза.

1.4.1 Microsoft Word и встроенные средства рецензирования

Текстовый редактор Microsoft Word широко применяется для подготовки и проверки документов. Он предоставляет следующие возможности:

- создание и редактирование документов;
- проверка орфографии и грамматики;
- режим рецензирования и внесения исправлений;
- использование шаблонов оформления.

Недостатком данного подхода является отсутствие автоматизированной проверки документа на соответствие локальным требованиям конкретного вуза. Проверка структуры, параметров оформления и соответствия внутренним стандартам выполняется вручную.

1.4.2 Google Docs

Google Docs является облачным сервисом для совместной работы с документами. Его возможности включают:

- онлайн-редактирование файлов;
- совместную работу нескольких пользователей;
- систему комментариев и предложений;
- хранение документов в облаке.

Несмотря на удобство совместного редактирования, сервис не предоставляет специализированных механизмов автоматической проверки учебно-методических материалов по требованиям университета, а также не содержит функций аналитической отчётности для РИО.

1.4.3 Системы электронного документооборота

Корпоративные системы электронного документооборота предназначены для регистрации, маршрутизации и хранения документов. Они позволяют:

- вести архив документов;
- организовывать согласование файлов;
- разграничивать права доступа;
- контролировать исполнение задач.

Однако подобные системы являются универсальными решениями и не включают инструменты интеллектуального анализа оформления документов DOCX, автоматического подчёркивания ошибок и генерации титульных листов.

1.4.4 Вывод по анализу аналогов

Проведённый анализ существующих решений показал, что универсальные офисные продукты и системы электронного документооборота способны решать лишь отдельные задачи редакционно-издательского отдела.

Одни системы обеспечивают редактирование документов, другие — хранение файлов и организацию согласования, однако ни одно из рассмотренных решений не сочетает в себе функции автоматической проверки оформления методических материалов, генерации отчётов об ошибках, формирования исправленного DOCX-файла и накопления статистики работы подразделения.

Следовательно, наиболее целесообразным является создание собственной специализированной информационно-аналитической системы, учитывающей особенности документооборота и нормативные требования конкретного высшего учебного заведения.

1.5 Постановка задач на разработку

На основании проведённого анализа были сформулированы следующие задачи, которые должна решать разрабатываемая система:

1. Обеспечить регистрацию и авторизацию пользователей с разграничением прав доступа.
2. Реализовать загрузку методических, учебных и научных материалов в формате DOCX.
3. Выполнить автоматическую проверку документов на соответствие установленным требованиям оформления.
4. Формировать отчёт с перечнем найденных нарушений.
5. Создавать копию документа DOCX с визуальным выделением ошибок.
6. Автоматически сохранять корректно оформленные работы в централизованной базе данных.
7. Реализовать хранение и поиск ранее загруженных материалов.
8. Предоставить средства формирования статистической и аналитической отчётности.
9. Реализовать модуль автоматического создания титульных листов по шаблону вуза.
10. Создать удобный веб-интерфейс для пользователей и административную панель для управления системой.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра «Информационно-аналитическая система для РИО вуза».

2.2 Цель и назначение разработки

Основной целью работы является разработка и внедрение информационно-аналитической системы для автоматизации процессов проверки, хранения и сопровождения учебно-методических и научных материалов, поступающих в редакционно-издательский отдел вуза.

Задачами разработки являются:

- анализ предметной области и выявление требований к разрабатываемой системе;
- проектирование архитектуры программного средства, базы данных и пользовательского интерфейса;
- разработка серверной и клиентской частей информационной системы;
- реализация и интеграция модулей автоматической проверки документов, хранения файлов и формирования отчётности;
- тестирование, внедрение и оценка эффективности разработанной системы.

2.3 Требования к функциональности

Информационная система должна обеспечивать выполнение следующих функций:

1. **Управление пользователями.** Система должна обеспечивать регистрацию и авторизацию пользователей по логину и паролю. Должно быть реализовано разграничение прав доступа: администратор, редактор, пользо-

ватель. Администратор должен иметь возможность управления учётными записями пользователей.

2. Загрузка документов. Пользователь должен иметь возможность загружать методические материалы, учебные пособия, монографии и иные документы в формате DOCX через веб-интерфейс системы.

3. Автоматическая проверка оформления. После загрузки документа система должна автоматически анализировать файл на соответствие установленным требованиям оформления. Проверка должна включать контроль структуры документа, параметров шрифта, интервалов, отступов, заголовков, таблиц, изображений, списков литературы и иных элементов.

4. Формирование отчёта об ошибках. По завершении анализа система должна формировать отчёт, содержащий перечень обнаруженных нарушений, описание ошибок и рекомендации по их устранению.

5. Создание документа с выделением ошибок. Система должна создавать копию загруженного документа в формате DOCX, в которой найденные ошибки визуально выделены средствами форматирования.

6. Хранение корректных документов. Если загруженный документ соответствует всем требованиям, он должен автоматически сохраняться в централизованной базе данных или файловом архиве системы.

7. Поиск и просмотр документов. Система должна обеспечивать просмотр списка ранее загруженных материалов, поиск по названию, автору, дате загрузки, статусу проверки и другим параметрам.

8. Генерация титульных листов. Пользователь должен иметь возможность заполнить форму с исходными данными, после чего система должна автоматически сформировать титульный лист по утверждённому шаблону вуза.

9. Статистика и аналитика. Система должна формировать сводные данные по количеству загруженных документов, числу успешных проверок, количеству найденных ошибок, активности пользователей и временным показателям обработки материалов.

10. Журналирование действий. Система должна сохранять сведения о действиях пользователей: загрузка файлов, проверка документов, вход в систему, генерация титульных листов и иные операции.

2.4 Требования к интерфейсу

Интерфейс информационной системы представляет собой веб-приложение с единым современным стилем оформления, обеспечивающее удобную и интуитивно понятную навигацию. Пользовательский интерфейс должен быть адаптирован для работы в распространённых браузерах и обеспечивать быстрый доступ к основным функциям системы.

Основные элементы интерфейса включают:

- главную страницу системы;
- форму авторизации и регистрации пользователей;
- личный кабинет пользователя;
- страницу загрузки документов на проверку;
- страницу просмотра результатов проверки;
- страницу архива сохранённых документов;
- страницу генерации титульных листов;
- административную панель управления пользователями;
- модуль просмотра статистики и аналитических данных.

На рисунке 2.1 представлена страница авторизации системы.

Как показано на рисунке, форма авторизации содержит поля ввода логина и пароля, кнопку входа в систему, а также элементы перехода к регистрации нового пользователя. После успешной авторизации пользователь получает доступ к функциональным возможностям системы в соответствии со своей ролью.

На рисунке 2.2 представлен внешний вид личного кабинета пользователя.

Проверка методичек Веб-система

Авторизация

Профиль

Проверка

Титульный лист

Статистика

Администрирование

Статус
Авторизован

Система автономной проверки файлов

Загрузка файлов, проверка оформления, PDF-отчёт и статистика

Обновить профиль Выйти

Авторизация

Вход

Email
do6rblubatman@mail.ru

Пароль

Войти

Регистрация

ФИО

Email
do6rblubatman@mail.ru

Пароль

Код факультета
09.03.04

Название кафедры

Роль
Доцент

Зарегистрироваться

Рисунок 2.1 – Страница авторизации

Проверка методичек Веб-система

Авторизация

Профиль

Проверка

Титульный лист

Статистика

Администрирование

Статус
Авторизован

Система автономной проверки файлов

Загрузка файлов, проверка оформления, PDF-отчёт и статистика

Обновить профиль Выйти

Профиль пользователя

ФИО Mikhail Petrov	Email do6rblubatman@mail.ru	Роль Доцент
Факультет ФФИПИ	Кафедра Программная инженерия	Создан 23.04.2026, 22:45:48

Рисунок 2.2 – Личный кабинет пользователя

Личный кабинет содержит навигационное меню, сведения о текущем пользователе, основные разделы системы, а также быстрый доступ к загрузке документов, архиву файлов и генерации титульных листов.

На рисунке 2.3 представлена страница проверки документов.

Система автономной проверки файлов
Загрузка файлов, проверка оформления, PDF-отчет и статистика

Обновить профиль Выйти

Система проверки файлов

Выберите .doc или .docx файл
Выберите файл | Файл не выбран

Проверить файл

Здесь появится результат проверки

Мои файлы Обновить список

Не удалось загрузить список методичек

Статус: Авторизован

Рисунок 2.3 – Страница проверки документов

Данная страница содержит форму выбора файла формата DOCX, кнопку запуска автоматической проверки, область отображения текущего статуса обработки и результаты анализа после завершения проверки.

На рисунке 2.4 представлена страница генерации титульных листов.

Система автономной проверки файлов
Загрузка файлов, проверка оформления, PDF-отчет и статистика

Обновить профиль Выйти

Генерация титульного листа

Тип титульного листа
Методические указания

Методические указания — первая страница

Название методички

Название дисциплины

Для кого
Студентов

Код направления
09 03 04

Название направления

Город
Курск

Год
2026

Методические указания — вторая страница

УДК

ФИО составителя

ФИО рецензента

Степень рецензента

Краткое описание
Вводите краткое описание

Не более 1000 символов 0 / 1000

Сгенерировать титульный лист

Рисунок 2.4 – Страница генерации титульных листов

Как показано на рисунке, страница содержит форму ввода реквизитов работы: наименование дисциплины, тема работы, сведения об авторе, руко-

водителе и кафедре. После заполнения формы пользователь может сформировать готовый документ в формате DOCX.

На рисунке 2.5 представлена страница статистики и аналитики.

Рисунок 2.5 – Страница статистики

Страница статистики содержит сведения о количестве зарегистрированных пользователей, числе загруженных документов, результатах проверок, количестве обнаруженных ошибок, а также иные аналитические показатели деятельности системы.

Интерфейс системы должен обеспечивать логичное расположение элементов управления, единообразие оформления страниц, наглядность представления результатов работы и минимальное количество действий пользователя при выполнении основных операций.

2.5 Требования к обработке ошибок

Система должна обеспечивать корректную обработку исключительных ситуаций.

Основные требования:

- при возникновении ошибки пользователю должно отображаться понятное сообщение;

- внутренние ошибки системы не должны раскрывать служебную информацию;
- ошибки должны фиксироваться в журнале событий;
- должна быть реализована обработка:
 - некорректных входных данных;
 - ошибок загрузки файлов;
 - ошибок обработки документов;
 - ошибок взаимодействия с базой данных.

2.6 Требования к удобству использования (UX)

Система должна обеспечивать удобство и простоту взаимодействия пользователей.

Основные требования:

- интерфейс должен быть интуитивно понятным;
- количество действий для выполнения основной операции (загрузка и проверка документа) должно быть минимальным;
- результаты проверки должны отображаться в структурированном виде;
- должна быть реализована визуальная обратная связь (индикаторы загрузки, статусы операций).

2.7 Моделирование вариантов использования

Для определения функциональных возможностей разрабатываемой системы и взаимодействия пользователей с её компонентами была выполнена модель вариантов использования.

В системе предусмотрены следующие роли пользователей:

Администратор — обладает полными правами доступа, включая управление пользователями, просмотр статистики, настройку системы, удаление документов и контроль работы пользователей.

Редактор — осуществляет просмотр загруженных материалов, контроль результатов автоматической проверки, работу с архивом документов и формирование отчётности.

Пользователь — выполняет загрузку документов, получает результаты проверки, скачивает отчёты, использует модуль генерации титульных листов и просматривает историю своих материалов.

Основные прецеденты (варианты использования) системы:

1. Вход в систему.
2. Регистрация пользователя.
3. Загрузка документа на проверку.
4. Автоматическая проверка оформления.
5. Просмотр отчёта об ошибках.
6. Скачивание исправленного документа.
7. Сохранение корректного документа в архив.
8. Поиск и просмотр ранее загруженных работ.
9. Генерация титульного листа.
10. Просмотр статистики работы системы.
11. Управление пользователями.
12. Выход из системы.

Для повышения наглядности диаграмма прецедентов была разделена по ролям пользователей. Отдельно представлены варианты использования для пользователя, администратора и редактора.

На рисунке 2.6 представлена диаграмма прецедентов пользователя.

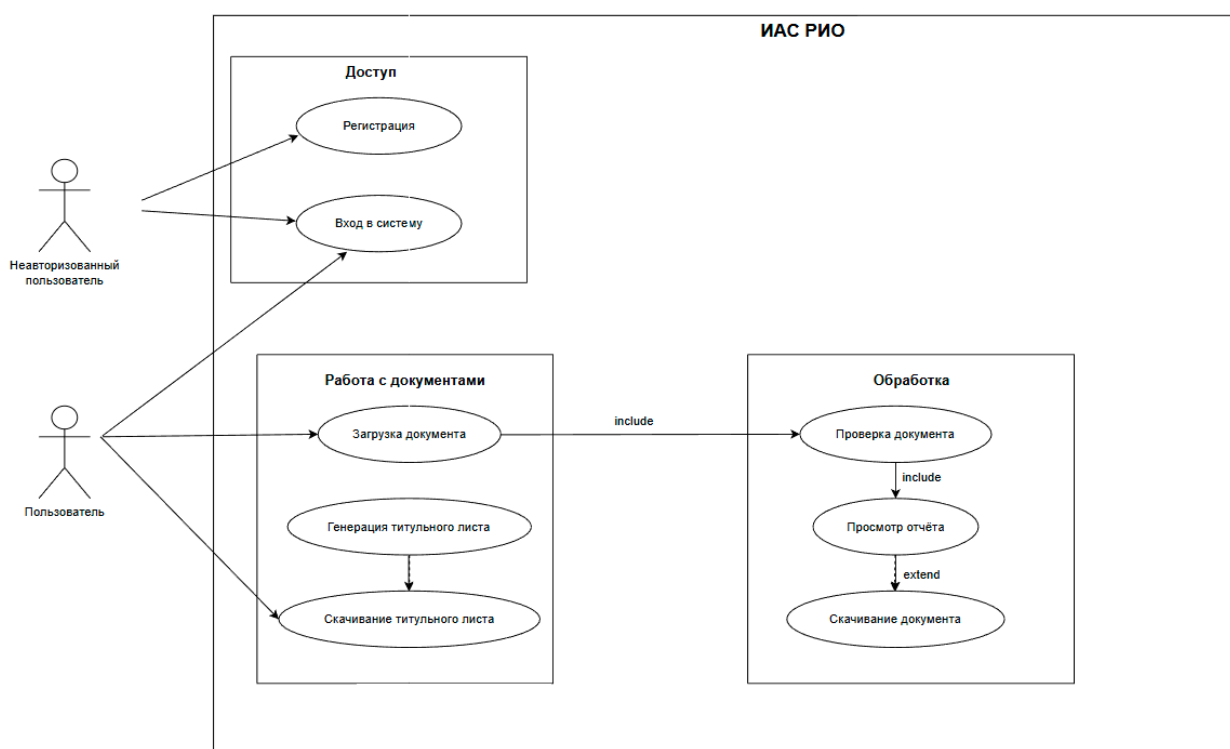


Рисунок 2.6 – Диаграмма прецедентов пользователя

На рисунке 2.7 представлена диаграмма прецедентов администратора.



Рисунок 2.7 – Диаграмма прецедентов администратора

На рисунке 2.8 представлена диаграмма прецедентов редактора.

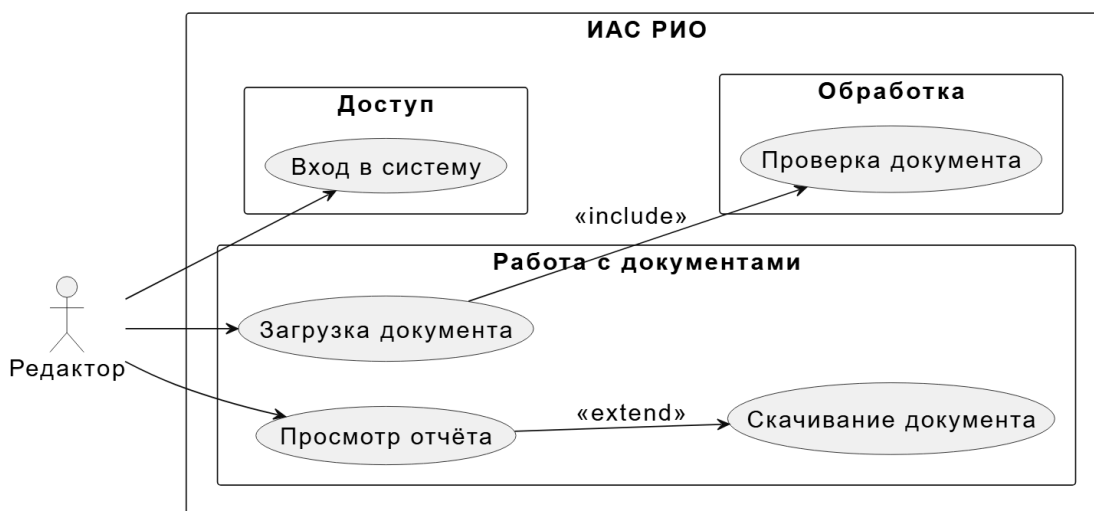


Рисунок 2.8 – Диаграмма прецедентов редактора

2.7.1 Вариант использования «Вход в систему»

Заинтересованные лица и их требования: пользователь желает получить доступ к функциям системы путём ввода логина и пароля.

Предусловие: пользователь не авторизован; открыта страница входа.

Постусловие: пользователь успешно авторизован и перенаправлен в личный кабинет либо на главную страницу системы.

Основной успешный сценарий:

1. Пользователь открывает страницу входа.
2. Вводит логин и пароль.
3. Нажимает кнопку «Войти».
4. Система выполняет поиск пользователя в базе данных.
5. Выполняется проверка пароля.
6. Создаётся пользовательская сессия.
7. Пользователь получает доступ к функциям системы.

Альтернативный сценарий (неверные данные):

1. Пользователь вводит неверный логин или пароль.
2. Система отображает сообщение об ошибке авторизации.
3. Пользователь остаётся на странице входа.

Альтернативный сценарий (пустые поля):

1. Пользователь не заполняет одно из обязательных полей.
2. Система выводит сообщение о необходимости заполнения формы.

2.7.2 Вариант использования «Выход из системы»

Заинтересованные лица и их требования: авторизованный пользователь желает завершить работу с системой.

Предусловие: пользователь авторизован.

Постусловие: пользовательская сессия завершена.

Основной успешный сценарий:

1. Пользователь нажимает кнопку «Выход».
2. Система удаляет активную сессию.
3. Пользователь перенаправляется на страницу входа.

2.7.3 Вариант использования «Загрузка документа на проверку»

Заинтересованные лица и их требования: пользователь желает отправить документ в формате DOCX для автоматической проверки.

Предусловие: пользователь авторизован.

Постусловие: файл загружен в систему и передан модулю проверки.

Основной успешный сценарий:

1. Пользователь открывает страницу загрузки.
2. Выбирает файл формата DOCX.
3. Нажимает кнопку «Загрузить».
4. Система сохраняет файл во временное хранилище.
5. Запускается процедура проверки документа.

Альтернативный сценарий (неподдерживаемый формат):

1. Пользователь выбирает файл иного формата.
2. Система выводит сообщение об ошибке.
3. Загрузка не выполняется.

Альтернативный сценарий (файл не выбран):

1. Пользователь нажимает кнопку загрузки без выбора файла.

2. Система предлагает выбрать документ.

2.7.4 Вариант использования «Автоматическая проверка документа»

Заинтересованные лица и их требования: пользователь желает получить результат проверки оформления документа.

Предусловие: документ успешно загружен.

Постусловие: сформирован результат проверки.

Основной успешный сценарий:

1. Система анализирует структуру документа.
2. Выполняется проверка параметров оформления.
3. Выявляются ошибки и несоответствия требованиям.
4. Формируется перечень найденных нарушений.

Альтернативный сценарий (ошибка обработки файла):

1. Документ повреждён либо содержит некорректную структуру.
2. Проверка прерывается.
3. Пользователь получает сообщение об ошибке обработки документа.

2.7.5 Вариант использования «Получение отчёта об ошибках»

Заинтересованные лица и их требования: пользователь желает ознакомиться с найденными нарушениями.

Предусловие: проверка завершена.

Постусловие: пользователю отображён отчёт.

Основной успешный сценарий:

1. После завершения проверки система формирует отчёт.
2. Пользователь открывает страницу результата.
3. Отображается список ошибок с пояснениями.

Альтернативный сценарий (ошибок не обнаружено):

1. Проверка завершена без замечаний.
2. Система отображает сообщение о корректности документа.

2.7.6 Вариант использования «Скачивание документа с выделенными ошибками»

Заинтересованные лица и их требования: пользователь желает скачать исправляемый файл с визуально отмеченными ошибками.

Предусловие: документ содержит нарушения.

Постусловие: пользователю предоставлен файл DOCX для скачивания.

Основной успешный сценарий:

1. Пользователь нажимает кнопку скачивания.
2. Система формирует копию документа.
3. Ошибки подчёркиваются или выделяются.
4. Пользователь скачивает файл.

Альтернативный сценарий (файл не сформирован):

1. Во время генерации файла возникает ошибка.
2. Система выводит сообщение о невозможности подготовки документа.

2.7.7 Вариант использования «Автоматическое сохранение корректного документа»

Заинтересованные лица и их требования: пользователь желает сохранить корректно оформленную работу в архиве системы.

Предусловие: ошибок в документе не обнаружено.

Постусловие: файл помещён в централизованное хранилище.

Основной успешный сценарий:

1. Проверка завершается без ошибок.
2. Система сохраняет документ в архив.
3. Информация о работе заносится в базу данных.

Альтернативный сценарий (ошибка сохранения):

1. Возникает ошибка доступа к хранилищу либо базе данных.
2. Система фиксирует ошибку в журнале событий.

3. Пользователю выводится уведомление.

2.7.8 Вариант использования «Просмотр архива документов»

Заинтересованные лица и их требования: пользователь желает просмотреть ранее сохранённые документы.

Предусловие: пользователь авторизован.

Постусловие: отображён список документов архива.

Основной успешный сценарий:

1. Пользователь открывает раздел архива.
2. Система загружает список документов.
3. Пользователь просматривает или скачивает нужный файл.

Альтернативный сценарий (архив пуст):

1. В системе отсутствуют сохранённые документы.
2. Отображается сообщение «Архив пуст».

2.7.9 Вариант использования «Формирование титульного листа»

Заинтересованные лица и их требования: пользователь желает автоматически создать титульный лист по шаблону вуза.

Предусловие: пользователь авторизован.

Постусловие: сформирован титульный лист.

Основной успешный сценарий:

1. Пользователь открывает форму генерации титульного листа.
2. Заполняет необходимые поля.
3. Нажимает кнопку формирования.
4. Система создаёт документ DOCX.

Альтернативный сценарий (не заполнены обязательные поля):

1. Пользователь оставляет обязательные поля пустыми.
2. Система сообщает о необходимости заполнения данных.

2.7.10 Вариант использования «Просмотр статистики»

Заинтересованные лица и их требования: администратор желает получить статистические сведения о работе системы.

Предусловие: администратор авторизован.

Постусловие: отображён аналитический отчёт.

Основной успешный сценарий:

1. Администратор открывает раздел статистики.
2. Система подсчитывает показатели.
3. Отображаются данные по количеству проверок, ошибок, сохранённых документов и пользователям.

Альтернативный сценарий (недостаточно прав):

1. Обычный пользователь пытается открыть раздел статистики.
2. Система запрещает доступ.

2.7.11 Вариант использования «Управление пользователями»

Заинтересованные лица и их требования: администратор желает управлять учётными записями пользователей.

Предусловие: администратор авторизован.

Постусловие: учётные записи обновлены.

Основной успешный сценарий:

1. Администратор открывает раздел пользователей.
2. Добавляет, редактирует или удаляет записи.
3. Система сохраняет изменения.

Альтернативный сценарий (дублирование логина):

1. При создании пользователя введён уже существующий логин.
2. Система выводит сообщение об ошибке.

2.7.12 Вариант использования «Настройка параметров системы»

Заинтересованные лица и их требования: администратор желает изменить правила проверки документов и параметры работы системы.

Предусловие: администратор авторизован.

Постусловие: новые параметры сохранены.

Основной успешный сценарий:

1. Администратор открывает настройки.
2. Изменяет параметры.
3. Подтверждает сохранение.
4. Система применяет новые значения.

Альтернативный сценарий (некорректные параметры):

1. Введены недопустимые значения настроек.
2. Система отменяет сохранение и выводит сообщение об ошибке.

2.8 Требования к надёжности

Разрабатываемая информационно-аналитическая система должна обеспечивать стабильную и корректную работу при различных режимах эксплуатации, а также гарантировать сохранность пользовательских данных.

Целостность данных. Все операции загрузки документов, сохранения результатов проверки, регистрации пользователей и формирования отчётов должны выполняться атомарно. При возникновении ошибки операция должна быть отменена без повреждения существующих данных.

Устойчивость к сбоям. При аварийном завершении работы сервера, отключении электропитания или программном сбое система должна сохранять ранее записанные данные. После повторного запуска система должна продолжить работу в штатном режиме.

Контроль корректности входных данных. На уровне приложения должна выполняться проверка корректности вводимых пользователями данных: обязательность заполнения полей, допустимые форматы файлов, ограничения длины строковых значений, корректность дат и иных параметров.

Логирование ошибок. Критические ошибки, исключения и события работы системы должны фиксироваться в журнале событий. Записи журнала должны содержать дату, время, уровень важности и описание ошибки.

Резервное копирование. База данных и архив документов должны допускать резервное копирование штатными средствами используемой СУБД и файловой системы. Восстановление данных должно выполняться из резервной копии.

2.9 Требования к информационной безопасности

Система должна обеспечивать защиту данных от несанкционированного доступа и разграничение прав пользователей.

Авторизация пользователей. Доступ к функциям системы должен предоставляться только после успешного ввода логина и пароля.

Хранение паролей. Пароли пользователей не должны храниться в открытом виде. Для хранения необходимо использовать криптографическое хеширование.

Разграничение прав доступа. В системе должны поддерживаться роли пользователей:

- пользователь — загрузка документов, просмотр результатов, генерация титульных листов;
- администратор — полный доступ, включая управление пользователями и просмотр статистики.

Проверка доступа на серверной стороне. Права пользователя должны проверяться сервером при выполнении каждого защищённого действия.

Защита от SQL-инъекций. Доступ к базе данных должен осуществляться с использованием ORM либо параметризованных запросов.

Защита файлового хранилища. Загруженные документы и архивные материалы должны храниться в каталогах, недоступных для прямого неавторизованного обращения.

2.10 Требования к программной и технической совместимости

Система должна функционировать в типовой вычислительной среде и не предъявлять завышенных требований к аппаратным ресурсам.

Операционная система сервера:

- Windows 10/11.

Аппаратные требования:

- процессор с тактовой частотой от 2 ГГц;
- оперативная память не менее 4 ГБ;
- свободное дисковое пространство от 1 ГБ без учёта пользовательских файлов.

Клиентская часть. Для работы пользователей необходимы текстовый редактор Word и современный веб-браузер:

- Яндекс Браузер;
- Google Chrome;
- Mozilla Firefox;
- Microsoft Edge.

2.11 Требования к оформлению документации

Разработка программной документации и программного изделия должна выполняться в соответствии с требованиями действующих нормативных документов ГОСТ 19.102–77 и ГОСТ 34.601–90. Единая система программной документации.

3 Технический проект

3.1 Архитектура программной системы

Разрабатываемая информационно-аналитическая система построена по архитектуре «клиент-сервер» с использованием REST-подхода.

Серверная часть реализована на языке Python с использованием фреймворка FastAPI и обеспечивает обработку запросов, выполнение бизнес-логики, а также взаимодействие с базой данных PostgreSQL и файловым хранилищем документов.

Клиентская часть представляет собой веб-приложение, доступ к которому осуществляется через браузер. Взаимодействие между клиентом и сервером выполняется по протоколу HTTP с использованием формата JSON для обмена данными.

Архитектура системы включает следующие основные компоненты:

- клиентское веб-приложение (интерфейс пользователя);
- серверное приложение (FastAPI);
- система управления базами данных PostgreSQL;
- файловое хранилище для загружаемых и сгенерированных документов;
- модуль анализа и проверки документов формата DOCX.

Логика работы системы организована следующим образом. Пользователь через веб-интерфейс выполняет действия (авторизация, загрузка документа, запуск проверки и др.), которые инициируют HTTP-запросы к серверу. Сервер обрабатывает запрос, при необходимости обращается к базе данных и файловой системе, выполняет проверку документа и возвращает результат клиенту.

Модуль проверки документов осуществляет анализ структуры и оформления файлов формата DOCX, выявляет несоответствия установленным требованиям и формирует отчёт об ошибках. При необходимости система генерирует исправленный файл с визуальным выделением нарушений.

Все загружаемые документы сохраняются в файловой системе, а информация о них (путь к файлу, результаты проверки, пользователь и др.) — в базе данных PostgreSQL. Это обеспечивает разделение данных и повышает масштабируемость системы.

Для обеспечения безопасности и управления доступом используется механизм аутентификации пользователей. После успешного входа пользователь получает токен доступа, который используется при последующих запросах к API.

Общая схема взаимодействия компонентов системы представлена на рисунке 3.1.

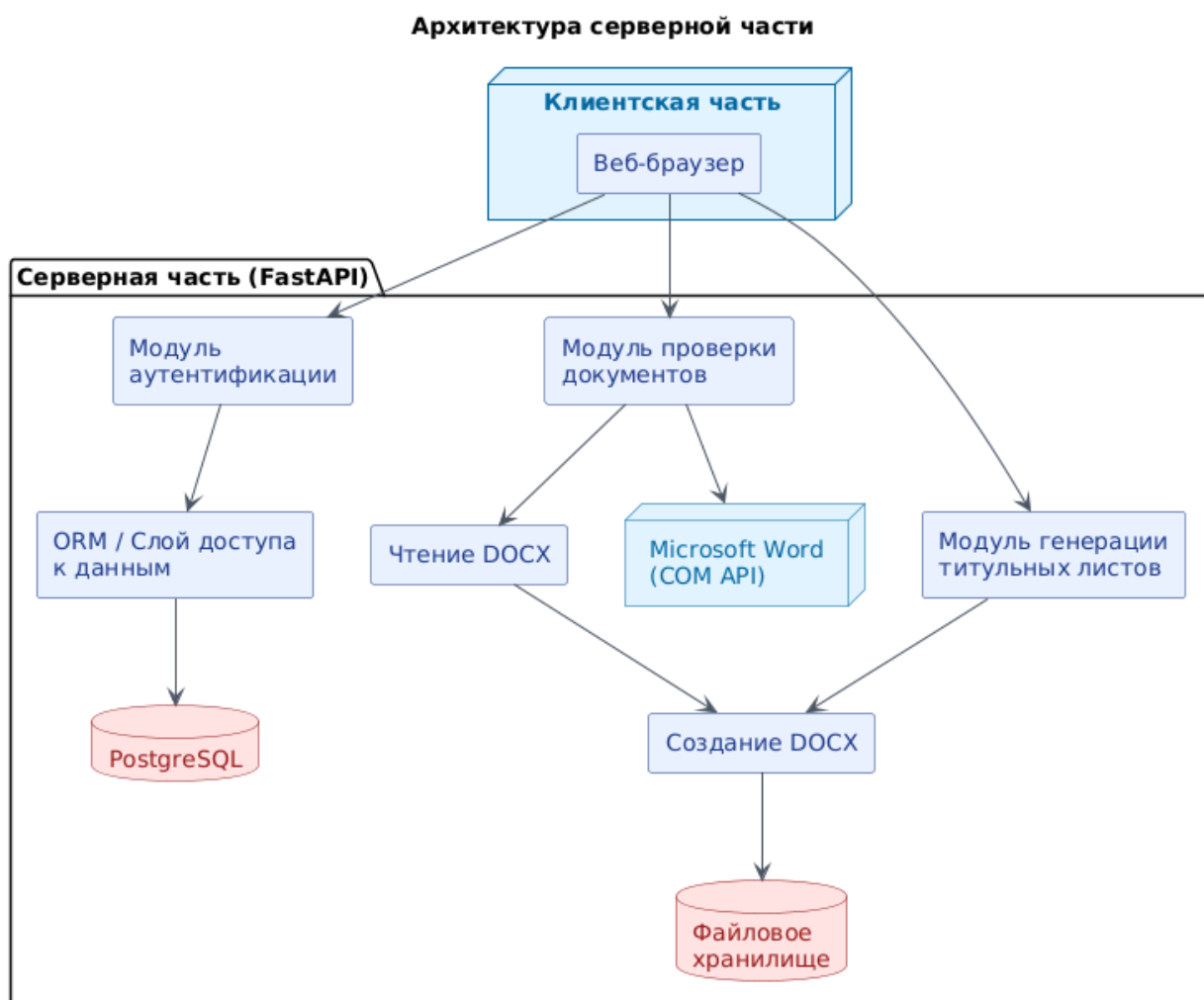


Рисунок 3.1 – Общая схема архитектуры системы

3.2 Модуль обработки учебно-методических документов

Модуль обработки учебно-методических документов является центральным компонентом системы, отвечающим за приём, анализ и проверку файлов формата DOCX. Он одинаково работает для методических указаний, учебных пособий и монографий, поскольку алгоритм проверки оформления и механизмы работы с документами универсальны. Различия между типами документов проявляются только на этапе генерации титульных листов, где используются разные наборы полей и шаблоны оформления.

3.2.1 Общая архитектура модуля

Модуль построен по многоуровневой архитектуре и включает следующие компоненты:

- **слой маршрутизации** — приём HTTP-запросов, валидация входных данных, маршрутизация к соответствующим обработчикам;
- **сервисный слой** — реализация бизнес-логики: проверка документов, генерация титульных листов, расчёт статистики;
- **слой доступа к данным** — объектно-реляционное отображение для взаимодействия с PostgreSQL;
- **слой утилит** — вспомогательные функции для работы с файлами, хеширования, логирования.

Такое разделение обеспечивает слабую связанность компонентов и упрощает тестирование и дальнейшее расширение.

3.2.2 Интеграция с Microsoft Word через COM-интерфейс

Ключевой особенностью модуля является использование COM-интерфейса Microsoft Word через библиотеку pywin32. При запуске проверки создаётся экземпляр приложения Word в фоновом режиме с отключёнными всплывающими окнами и диалогами. Это позволяет выполнять анализ и модификацию документов без видимого запуска графического интерфейса Word.

Основные операции, выполняемые через СОМ-интерфейс:

- открытие документа (`Documents.Open`) с возможностью только чтения или с правом записи;
- получение общей информации о документе: количество страниц, разделов, абзацев;
- обход всех абзацев документа и извлечение параметров форматирования;
- доступ к структуре страниц (поля, колонтитулы, нумерация);
- модификация документа: изменение цвета шрифта для выделения ошибок, добавление комментариев;
- сохранение изменённой копии и закрытие документа.

3.2.3 Алгоритм автоматической проверки оформления

Процесс проверки запускается после получения файла и его временно-го сохранения. Алгоритм реализован в классе `WordMethodicalChecker` и включает следующие этапы.

Этап 1. Предварительная обработка.

Файл сохраняется во временную директорию, создаётся уникальный идентификатор задачи для отслеживания прогресса. Если требуется сохранение исправленной версии (параметр `mark_document = True`), создаётся копия документа с суффиксом `_checked`, с которой будет работать модуль. Исходный файл остаётся неизменным.

Этап 2. Определение объёма работы.

Через СОМ-интерфейс открывается документ, вычисляется общее количество страниц с помощью метода `ComputeStatistics(WD_STATISTIC_PAGES)`. Эта информация используется для последующей группировки ошибок по страницам.

Этап 3. Фильтрация служебных элементов.

При обходе абзацев пропускаются:

- пустые абзацы — определяются по длине текста (менее 3 символов);

– абзацы, находящиеся внутри таблиц — метод `p.Range.Information(WD_WITHIN_TABLE)` возвращает `True` для таких элементов;

– заголовки — определяются по имени стиля: если в названии стиля присутствует подстрока "heading" или "заголов", абзац считается заголовком и пропускается.

Отфильтрованные элементы не участвуют в сравнении параметров оформления, так как к ним предъявляются иные требования или они не являются частью основного текста.

Этап 4. Извлечение параметров форматирования.

Для каждого значимого абзаца извлекаются следующие характеристики:

- **шрифт** — `p.Range.Font.Name`, эталонное значение "Times New Roman";
- **размер шрифта** — `p.Range.Font.Size`, эталонное значение 16 pt;
- **отступ первой строки** — `p.Range.ParagraphFormat.FirstLineIndent`, сначала проверяется формат самого абзаца, если значение близко к нулю, выполняется попытка получить отступ из стиля абзаца;
- **межстрочный интервал** — `p.Range.ParagraphFormat.LineSpacingRule`, эталонное значение `WD_LINE_SPACE_SINGLE` (одинарный интервал);
- **выравнивание текста** — анализируется при проверке номеров страниц.

Особое внимание уделяется случаю, когда в одном абзаце встречается неоднородное форматирование (например, разные размеры шрифта в одной строке). В таких ситуациях выполняется дополнительный обход всех слов внутри абзаца через коллекцию `p.Range.Words`. Для каждого слова извлекается его размер шрифта `w.Font.Size`. Если размер отличается от эталонного значения 16 pt более чем на 0,2 pt, и при этом слово не является знаком препинания (точка, запятая, скобка и т.п.), фиксируется ошибка «неверный размер шрифта» с указанием конкретного слова и номера страницы. Значения размера, равные 9999999,0, интерпретируются как ошибочные (такое значение возникает при попытке прочитать размер шрифта у объекта, не имеющего единого форматирования) и также приводят к фиксации нарушения. Такой

подход позволяет выявить ситуации, когда большая часть абзаца отформатирована правильно, но отдельные слова имеют неверный кегль, что часто встречается при копировании текста из внешних источников.

Этап 5. Сравнение с эталонами и фиксация ошибок.

Полученные параметры преобразуются в сантиметры (для отступов) и сравниваются с эталонными значениями:

- для отступа используется допуск 0,05 см: $\text{abs}(\text{indent_cm} - 1.25) > 0.05$;
- для размера шрифта используется допуск 0,2 pt: $\text{abs}(\text{size} - 16) > 0.2$;
- для шрифта и интервала выполняется точное сравнение (без допуска).

При обнаружении отклонения фиксируется номер страницы, тип нарушения и поясняющее сообщение. Номер страницы получается методом `p.Range.Information(WD_ACTIVE_END_PAGE_NUMBER)`. Каждая ошибка сохраняется в виде объекта `CheckIssue`, содержащего:

- `rule` — раздел проверки (например, "3" — для параметров текста, "4" — для нумерации);
- `severity` — степень важности ("ERROR" или "WARNING");
- `location` — местоположение в документе (страница или диапазон страниц);
- `message` — текст сообщения с описанием нарушения и рекомендацией по исправлению;
- `priority` — приоритет (используется для сортировки).

Этап 6. Проверка полей страницы.

После обработки всех абзацев выполняется проверка параметров страниц. Для каждого раздела документа (коллекция `doc.Sections`) извлекаются и проверяются настройки `PageSetup`:

- верхнее поле (`TopMargin`) — эталон 3,0 см;
- нижнее поле (`BottomMargin`) — эталон 2,25 см;
- левое поле (`LeftMargin`) — эталон 2,2 см;
- правое поле (`RightMargin`) — эталон 2,3 см.

Все поля проверяются в сантиметрах с допуском 0,05 см. При нарушении фиксируется номер раздела, и ошибка добавляется в отчёт.

Этап 7. Проверка нумерации страниц.

Проверка выполняется по всем разделам документа. Для каждого раздела анализируются верхний (Headers(WD_HEADER_FOOTER_PRIMARY)) и нижний (Footers(WD_HEADER_FOOTER_PRIMARY)) колонтитулы:

- Если в верхнем колонтитуле обнаружены поля с типом WD_FIELD_PAGE — нумерация присутствует в верхней части страницы, дополнительно проверяется выравнивание номера (должно быть по центру, WD_ALIGN_CENTER).

- Если номер найден только в нижнем колонтитуле — фиксируется ошибка о неверном расположении номера страницы.

- Если номер не найден ни в верхнем, ни в нижнем колонтитуле — фиксируется ошибка об отсутствии нумерации страниц.

Этап 8. Группировка ошибок.

Для повышения удобочитаемости отчёта последовательные страницы с одинаковыми типами ошибок объединяются в диапазоны. Например, если ошибки шрифта обнаружены на страницах 5, 6, 7, 8, отчёт покажет «Стр. 5–8» вместо четырёх отдельных записей. Группировка выполняется методом `_group_pages`, который сортирует номера страниц, находит в них непрерывные последовательности и формирует соответствующие записи. Для каждого диапазона создаётся одна запись в отчёте, а для всех страниц диапазона выполняется визуальное выделение ошибок.

Этап 9. Завершение проверки и освобождение ресурсов.

По окончании обхода всех абзацев и проверки полей и нумерации страниц выполняется сохранение изменений в документе (если `mark_document = True`) и закрытие документа через `doc.Close(False)`. После этого экземпляр Word завершается вызовом `self.word.Quit(False)`. Все временные файлы, созданные в процессе проверки (временная копия документа, файлы для отчёта), удаляются. Если на любом из этапов произошла ошибка, Word принудительно за-

вершается, а временные файлы удаляются в блоке `finally`, что гарантирует отсутствие «висящих» процессов и освобождение дискового пространства.

3.2.4 Формирование исправленной копии документа

Если в документе обнаружены ошибки и установлен флаг `mark_document` (активируется по умолчанию), модуль создаёт копию, в которой проблемные фрагменты визуально выделены. Алгоритм выделения работает следующим образом:

1. Для каждой группы ошибок определяется диапазон страниц, на которых они обнаружены.
2. Для первой страницы диапазона вычисляется начальная позиция через `doc.GoTo(1, 1, page)`. Константа 1 обозначает переход к странице, второй параметр — тип объекта (страница).
3. Для последней страницы диапазона вычисляется конечная позиция через `doc.GoTo(1, 1, page + 1)`. Если страница последняя в документе, конечная позиция определяется как `doc.Content.End`.
4. Область выделения задаётся через `doc.Range(start, end)`.
5. Цвет шрифта в выделенной области устанавливается в красный (`WD_COLOR_RED`) с помощью свойства `rng.Font.Color`.
6. К выделенной области добавляется комментарий с поясняющим сообщением через `doc.Comments.Add(rng, message)`.

Такой подход позволяет сохранить исходную структуру документа и одновременно наглядно показать автору все места, требующие исправления. Комментарии Word видны при наведении курсора и не нарушают вёрстку документа.

3.2.5 Генерация PDF-отчёта

Параллельно с модификацией документа формируется PDF-отчёт с использованием библиотеки `reportlab`. Процесс создания отчёта включает следующие шаги:

1. Регистрация шрифтов семейства Times New Roman (обычный, жирный, курсив, жирный курсив) для корректного отображения кириллицы. Библиотека reportlab не поддерживает кириллицу по умолчанию, поэтому требуется явная регистрация шрифтов с указанием пути к файлам .ttf.

2. Настройка параметров страницы: формат A4, поля 15 мм со всех сторон.

3. Формирование заголовочной части: название документа («Отчёт по проверке»), имя проверенного файла, дата и время проверки.

4. Определение итогового статуса:

- если есть ошибки уровня ERROR — статус «документ содержит критические ошибки» (отображается красным цветом);

- если есть только предупреждения (WARNING) — «документ содержит предупреждения» (оранжевый);

- если ошибок нет — «документ соответствует требованиям» (зелёный).

5. Построение таблицы ошибок с колонками: номер ошибки, раздел проверки, тип (ошибка/предупреждение), местоположение в документе, описание проблемы.

6. Стилизация таблицы: строки с ошибками выделяются красным фоном (BackgroundColor = #FEE2E2), строки с предупреждениями — жёлтым (BackgroundColor = #FEF3C7).

Отчёт сохраняется в директорию reports/ с именем, содержащим уникальный идентификатор, чтобы избежать конфликтов при одновременной работе нескольких пользователей.

3.2.6 Генерация титульных листов

Генерация титульных листов реализована через библиотеку python-docx без использования Microsoft Word. Это позволяет выполнять операцию быстро и без зависимости от установленного офисного пакета. В зависимости от типа документа используются разные наборы полей и шаблоны:

1. Для методических указаний: название, дисциплина, направление подготовки, составитель, рецензенты (с возможностью добавления нескольких), УДК, описание.

2. Для учебных пособий: автор, название, рецензенты, направления подготовки, значение А, ISBN, УДК, ББК, описание.

3. Для монографии: авторы (с возможностью добавления нескольких), название, город, год, УДК, ББК, ISBN, описание.

Алгоритм генерации единый для всех типов:

1. Создаётся новый пустой документ DOCX.

2. Устанавливаются поля страницы: верхнее 3 см, нижнее 2,25 см, левое 2,2 см, правое 2,3 см.

3. Задаётся стиль Normal: шрифт Times New Roman, размер 16 pt.

4. Последовательно добавляются абзацы с заданным выравниванием (центр, право, лево) и форматированием.

5. Для каждого абзаца через вспомогательную функцию `add_paragraph` устанавливаются параметры: текст, выравнивание, размер шрифта, жирное начертание, отступы до и после абзаца, межстрочный интервал.

6. После заполнения первой страницы добавляется разрыв раздела (`doc.add_section(WD_SECTION_START.NEW_PAGE)`), и формируется вторая страница.

7. Вторая страница содержит библиографическое описание, аннотацию и выходные данные — в зависимости от типа документа набор полей различается.

8. Готовый документ сохраняется во временную директорию, после чего отправляется пользователю и удаляется с сервера.

3.2.7 Управление сессиями и обновление токенов

Для поддержания авторизации пользователя без многократного ввода пароля используется двухуровневая система токенов: `access`-токен (срок действия 60 минут) и `refresh`-токен (срок действия 30 дней). `Access`-токен подписывается секретным ключом и содержит идентификатор пользователя и его

email. Refresh-токен генерируется как случайная строка, его хеш сохраняется в базе данных вместе с метаданными (IP-адрес, User-Agent).

При получении запроса к защищённому ресурсу сервер проверяет access-токен: если он истёк, клиент получает ошибку 401 Unauthorized и автоматически отправляет refresh-токен на эндпоинт /auth/refresh. Сервер проверяет, что refresh-токен не отозван (поле revoked), не истёк и принадлежит тому же пользователю, после чего выдаёт новый access-токен. Refresh-токен при этом остаётся без изменений, что позволяет обновлять access-токен многократно без повторного входа пользователя.

При выходе из системы соответствующий refresh-токен отмечается в базе данных как отозванный (revoked = True), что делает его непригодным для дальнейшего использования. Это позволяет администратору в случае подозрения на компрометацию учётной записи отозвать все токены пользователя одной командой.

3.2.8 Взаимодействие с базой данных через ORM

Для взаимодействия с PostgreSQL используется объектно-реляционное отображение SQLAlchemy. ORM позволяет работать с записями базы данных как с обычными объектами Python, скрывая детали формирования SQL-запросов. Каждая таблица описывается отдельным классом, наследующим от Base, с указанием типов полей, ограничений и связей.

Пример описания таблицы manual:

```
class Manual(Base):
    __tablename__ = "manual"
    id_manual = Column(Integer, primary_key=True, index=True)
    manual_name = Column(String(255), nullable=False)
    fio_user = Column(String(255), nullable=False)
    faculty_code = Column(String(10), nullable=False)
    department_name = Column(String(255), nullable=False)
    file_hash = Column(String(64), unique=True, nullable=False)
    created_at = Column(DateTime, server_default=func.now())
```


Сессии SQLAlchemy управляются через контекстный менеджер: при каждом запросе создаётся новая сессия, которая автоматически закрывается после окончания обработки. Это гарантирует, что соединения с базой данных не «утекают» и корректно возвращаются в пул.

Все запросы к базе данных используют параметризацию (через ORM-методы `filter` и `filter_by`), что полностью исключает возможность SQL-инъекций. При сохранении документа с успешно пройденной проверкой хеш файла проверяется на уникальность; если запись с таким хешем уже существует, операция вставки не выполняется, и пользователь получает соответствующее уведомление.

3.2.9 Предотвращение дублирования

Для исключения повторной загрузки одинаковых документов используется вычисление хеш-суммы файла (алгоритм SHA-256). Хеш вычисляется блоками по 8192 байта, что позволяет обрабатывать файлы любого размера без полной загрузки в память.

При успешной проверке (документ не содержит ошибок) хеш сохраняется в базе данных вместе с метаданной: название документа, ФИО автора, код факультета, название кафедры, дата загрузки. При каждой новой загрузке система выполняет следующие действия:

1. Вычисляется хеш загружаемого файла.
2. Выполняется запрос к базе данных на поиск записи с таким же хешем.
3. Если запись найдена, загрузка отклоняется, пользователю возвращается сообщение «Данный документ уже был загружен ранее».
4. Если запись не найдена, документ сохраняется, и в базу добавляется новая запись.

Это предотвращает накопление дубликатов и экономит дисковое пространство.

3.2.10 Обработка ошибок и восстановление

При работе с COM-интерфейсом Microsoft Word возможны различные сбои: документ может быть повреждён, Word может зависнуть или завершиться аварийно. В модуле реализована многоуровневая обработка исключений:

1. На уровне COM-вызовов — каждый вызов метода Word обернут в блок try-except. При возникновении ошибки (например, попытка открыть повреждённый файл) выполняется корректное закрытие документа и завершение процесса Word через `self.word.Quit(False)`. Пользователь получает сообщение с указанием причины ошибки и предложением проверить файл.

2. На уровне проверки документа — при сбое на любом этапе (фильтрация абзацев, извлечение параметров, группировка ошибок) проверка прерывается, пользователю возвращается код ошибки 500 Internal Server Error с пояснением. Временные файлы удаляются в блоке finally, чтобы не засорять дисковое пространство.

3. На уровне базы данных — при нарушении уникальности (попытка вставить дубликат) или других ограничениях целостности формируется понятное сообщение об ошибке без раскрытия деталей SQL-запроса.

Дополнительно реализован механизм тайм-аутов: если проверка документа занимает более 600 секунд (10 минут), операция прерывается, и пользователь получает сообщение о необходимости уменьшить размер файла или разбить его на части.

3.2.11 Хранение данных

Для хранения метаданных используется таблица `manual` в PostgreSQL, содержащая следующие поля:

- `id_manual` — первичный ключ (автоинкремент);
- `manual_name` — название документа;
- `fio_user` — ФИО автора;
- `faculty_code` — код факультета;

- department_name — название кафедры;
- file_hash — уникальный хеш (SHA-256);
- created_at — дата и время загрузки.

Сами файлы хранятся в файловой системе в директориях checked_files/ (проверенные документы с выделенными ошибками) и reports/ (PDF-отчёты). В базе хранятся только пути к этим файлам. Такой подход снижает нагрузку на СУБД и позволяет хранить файлы любого размера без ограничений.

3.2.12 Логирование

Все операции модуля логируются с использованием встроенного модуля logging. Настроены два уровня логирования:

1. INFO — фиксируются штатные операции: загрузка файла, успешное завершение проверки, генерация титульного листа, отправка результата пользователю, авторизация и выход пользователя.

2. ERROR — фиксируются сбои: невозможность открыть документ через COM-интерфейс, ошибка при сохранении файла, проблема с подключением к базе данных, тайм-аут проверки.

Логи сохраняются в файл с ротацией по дате, что позволяет администратору системы анализировать причины отказов, отслеживать подозрительную активность и своевременно устранять неполадки.

3.2.13 Заключение

Разработанный модуль обработки учебно-методических документов реализует полный цикл работы с документами формата DOCX: приём файлов, автоматическую проверку оформления через COM-интерфейс Microsoft Word, группировку ошибок, формирование исправленной копии с визуальной разметкой, генерацию PDF-отчётов, создание титульных листов через python-docx (с различными шаблонами для методических указаний, учебных пособий и монографий), управление сессиями через двухуровневую систему токенов, предотвращение дублирования с помощью хеширования, надёжное хранение метаданных в PostgreSQL, а также многоуровневую обработку

ошибок и логирование. Все компоненты модуля спроектированы с учётом требований к производительности, расширяемости и отказоустойчивости.

3.3 База данных и её проектирование

В качестве системы управления базами данных выбрана PostgreSQL. Эта СУБД обеспечивает высокую надёжность хранения информации, полную поддержку транзакций, соблюдение принципов ACID, а также располагает встроенными механизмами для контроля целостности данных на уровне ограничений и ссылочной целостности.

Подключение к базе данных реализовано через отдельный модуль `database.py`, который отвечает за конфигурацию параметров соединения, инициализацию подключений и их корректное завершение. В приложении задействован пул соединений — это позволяет переиспользовать уже установленные подключения, снижая накладные расходы на установку нового соединения для каждого запроса, и эффективно обслуживать большое количество одновременно работающих пользователей.

Для описания структур данных применяется объектно-реляционное отображение (ORM) SQLAlchemy. Каждая сущность предметной области представлена в коде отдельным классом, который описывает схему соответствующей таблицы: перечень полей, их типы, ограничения, а также связи с другими таблицами (один-ко-многим, многие-ко-многим). Такой подход упрощает взаимодействие с базой данных и снижает вероятность ошибок при формировании SQL-запросов.

3.3.1 ER-модель данных

На основе анализа предметной области были выделены основные сущности системы и связи между ними. ER-диаграмма отражает структуру данных и взаимосвязи между пользователями, кафедрами, факультетами и загруженными документами.

ER-диаграмма базы данных представлена на рисунке 3.2.

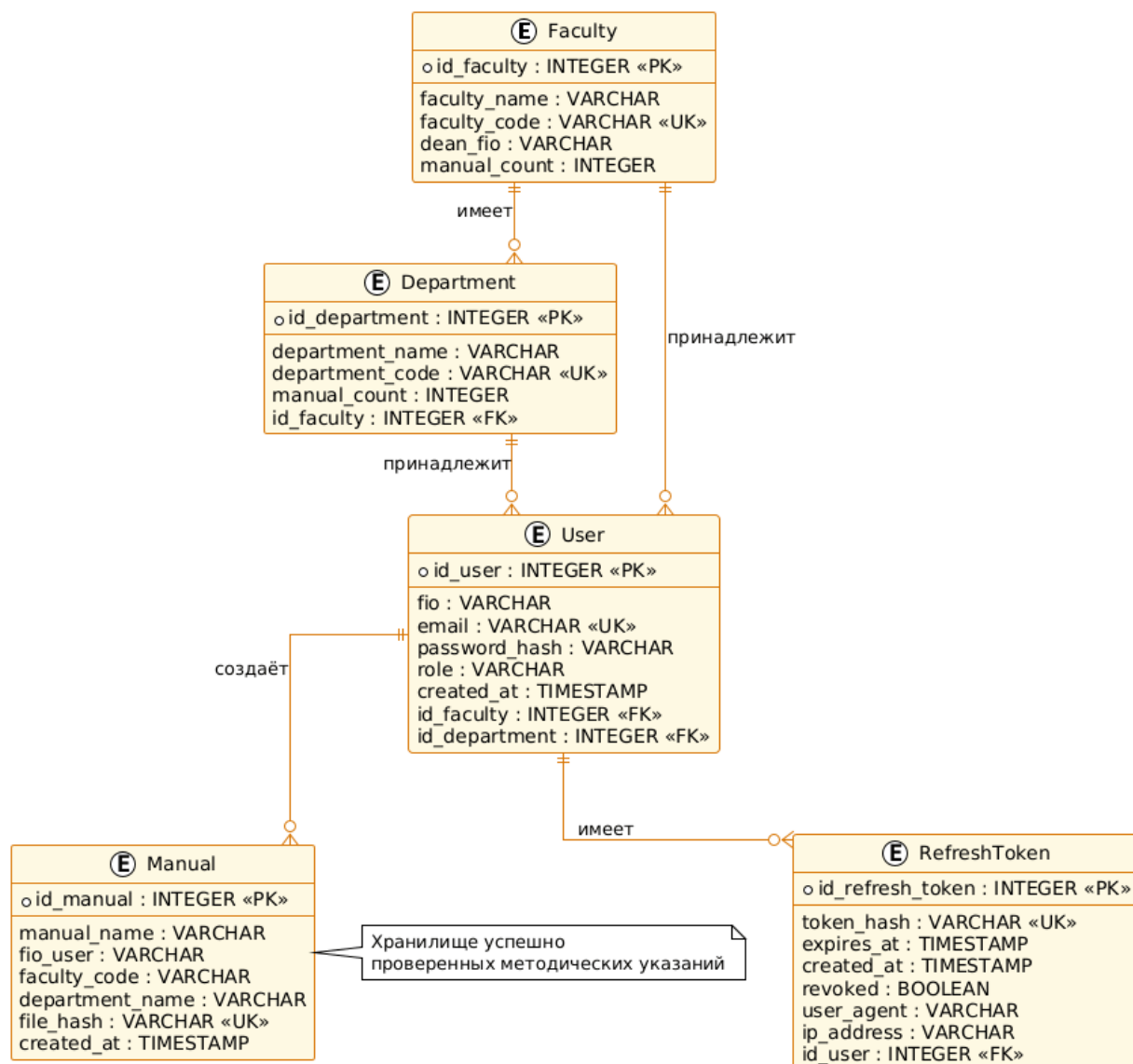


Рисунок 3.2 – ER-диаграмма базы данных

В модели центральное место занимает сущность «manual», связанная с пользователями и организационной структурой (кафедры и факультеты). Также выделена сущность токенов обновления, обеспечивающая механизм аутентификации пользователей.

К числу основных сущностей базы данных относятся:

- пользователи (users) — содержат учётные записи лиц, работающих с системой (авторы, преподаватели, сотрудники);
- факультеты (faculty) — подразделения университета, к которым привязаны пользователи и кафедры;
- кафедры (department) — структурные единицы внутри факультетов;

- документы (manual) — таблица, хранящая сведения о всех типах учебно-методических материалов (методические указания, учебные пособия, монографии), успешно прошедших проверку;
- токены обновления (refresh_tokens) — используются для поддержания сеансов авторизации.

3.3.2 Нормализация

Структура базы данных спроектирована в соответствии с требованиями третьей нормальной формы.

Каждая таблица оснащена первичным ключом, все поля содержат неделимые значения, а взаимосвязи между сущностями организованы через механизм внешних ключей. Избыточность данных устранена путём декомпозиции информации на логически независимые таблицы (факультеты, кафедры, пользователи).

Помимо этого, в схеме предусмотрены дополнительные ограничения, гарантирующие целостность данных. К ним относятся уникальные индексы (например, для адреса электронной почты пользователя и хеш-суммы файла документа), а также проверочные условия (например, допустимые значения для роли пользователя).

3.3.3 Реляционная модель данных

На основе ER-модели построена реляционная модель базы данных, отражающая структуру таблиц и их взаимосвязи.

Схема реляционной модели представлена на рисунке 3.3.

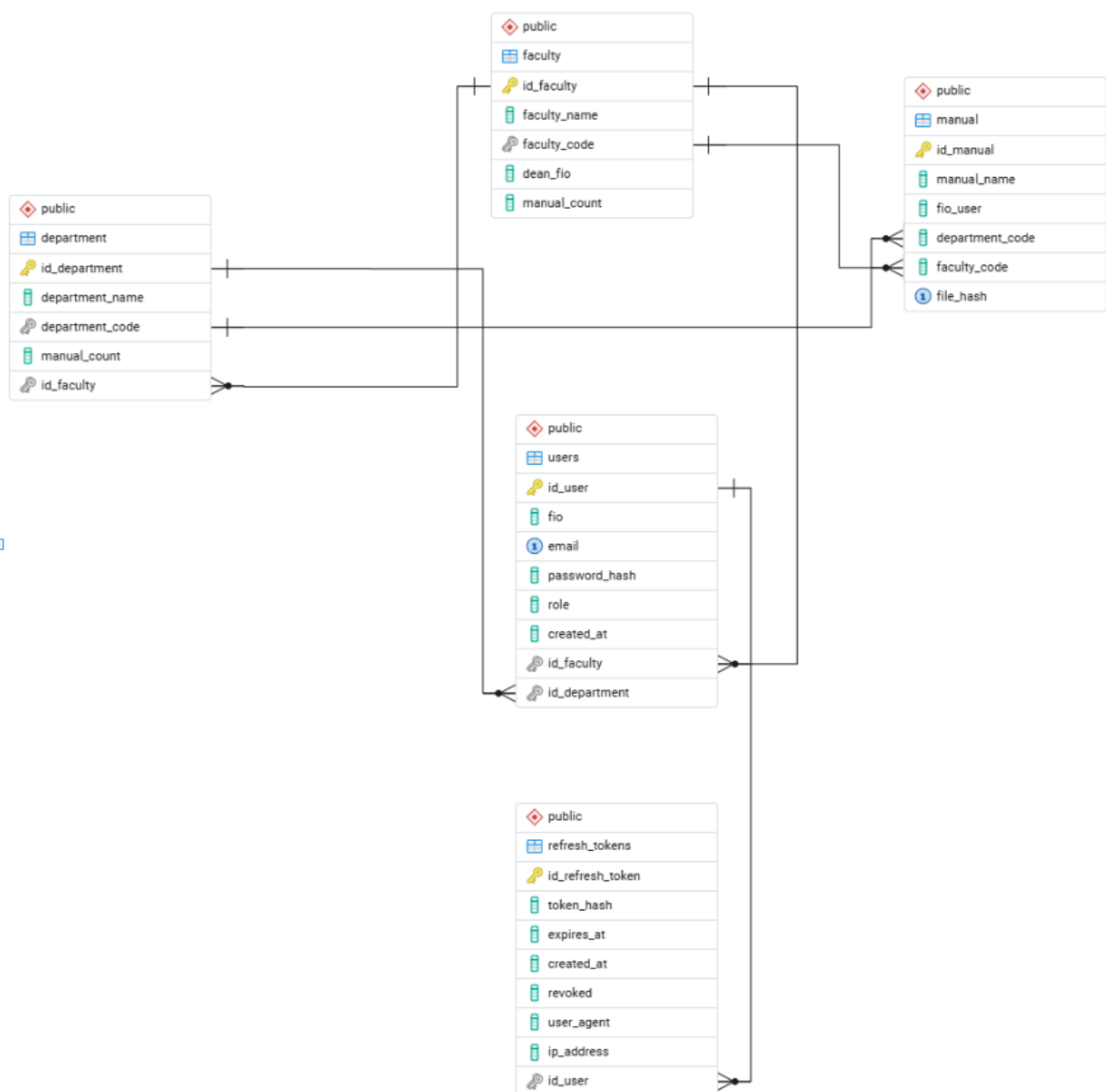


Рисунок 3.3 – Реляционная модель базы данных

Реляционная модель включает таблицы пользователей, факультетов, кафедр, документов (методических указаний, учебных пособий, монографий) и токенов обновления. Связи между таблицами реализованы через внешние ключи, что обеспечивает целостность данных и согласованность структуры.

Особое значение имеет таблица «manual», используемая для хранения информации о всех загружаемых документах и предотвращения их дублирования.

3.3.4 Описание структуры базы данных

Центральное место в базе данных занимает таблица «manual», предназначенная для хранения сведений о загружаемых документах независимо от их типа. Для каждой записи фиксируются следующие данные: название документа, фамилия, имя, отчество автора, код факультета, название кафедры, а также уникальная хеш-сумма файла. Хеш вычисляется на основе содержимого документа и служит для предотвращения дублирования — повторная загрузка одного и того же файла не приведёт к созданию лишней записи в базе.

Связь пользователей с организационной структурой обеспечивается через таблицы факультетов и кафедр. Каждый пользователь закреплён за конкретной кафедрой и факультетом, что позволяет отслеживать принадлежность авторов документов к соответствующим подразделениям и при необходимости формировать статистику по кафедрам или факультетам.

Файлы документов размещаются непосредственно в файловой системе сервера — это упрощает работу с большими объёмами данных и обеспечивает быстрый доступ к содержимому. В базе данных при этом сохраняются только метаданные (название, автор, дата загрузки) и уникальные идентификаторы, необходимые для поиска файла в файловом хранилище. Такое разделение позволяет снизить нагрузку на СУБД и эффективно масштабировать систему по мере увеличения количества загружаемых документов.

В таблицах 3.1–3.5 приведено подробное описание структуры всех таблиц базы данных.

Таблица 3.1 – Атрибуты таблицы «faculty» (Факультеты)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_faculty	SERIAL	
	*	faculty_name	VARCHAR	255
UK	*	faculty_code	INTEGER	
	*	dean_fio	VARCHAR	255

Продолжение таблицы 3.1

Key Type	Optionality	Column name	Data Type	Size
		manual_count	INTEGER	

Таблица 3.2 – Атрибуты таблицы «department» (Кафедры)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_department	SERIAL	
	*	department_name	VARCHAR	255
UK	*	department_code	INTEGER	
		manual_count	INTEGER	
FK	*	id_faculty	INTEGER	

Таблица 3.3 – Атрибуты таблицы «users» (Пользователи)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_user	SERIAL	
	*	fio	VARCHAR	255
UK	*	email	VARCHAR	255
	*	password_hash	VARCHAR	255
	*	role	VARCHAR	50
		created_at	TIMESTAMP	
FK	*	id_faculty	INTEGER	
FK	*	id_department	INTEGER	

Таблица 3.4 – Атрибуты таблицы «manual» (Документы)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_manual	SERIAL	
	*	manual_name	VARCHAR	255
	*	fio_user	VARCHAR	255

Продолжение таблицы 3.4

Key Type	Optionality	Column name	Data Type	Size
	*	department_code	INTEGER	
	*	faculty_code	INTEGER	
UK	*	file_hash	VARCHAR	64

Таблица 3.5 – Атрибуты таблицы «refresh_tokens» (Токены обновления)

Key Type	Optionality	Column name	Data Type	Size
PK	*	id_refresh_token	SERIAL	
UK	*	token_hash	VARCHAR	255
	*	expires_at	TIMESTAMP	
		created_at	TIMESTAMP	
	*	revoked	BOOLEAN	
		user_agent	VARCHAR	500
		ip_address	VARCHAR	100
FK	*	id_user	INTEGER	

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лутц, М. Изучаем Python. Том 1 / М. Лутц. – 5-е изд. – Санкт-Петербург : Питер, 2019. – 832 с. – ISBN 978-5-907144-52-1. – Текст : непосредственный.
2. Рамальо, Л. Python. К вершинам мастерства / Л. Рамальо. – Москва : ДМК Пресс, 2017. – 768 с. – ISBN 978-5-97060-384-3. – Текст : непосредственный.
3. Python Documentation : сайт. – URL: <https://docs.python.org/3/> (дата обращения: 24.04.2026). – Текст : электронный.
4. PEP 8 – Style Guide for Python Code : сайт. – URL: <https://peps.python.org/pep-0008/> (дата обращения: 24.04.2026). – Текст : электронный.
5. Параллельное программирование в Python (threading) : сайт. – URL: <https://docs.python.org/3/library/threading.html> (дата обращения: 24.04.2026). – Текст : электронный.
6. Logging HOWTO : сайт. – URL: <https://docs.python.org/3/howto/logging.html> (дата обращения: 24.04.2026). – Текст : электронный.
7. Буэно, С. Разработка API с FastAPI / С. Буэно. – Москва : ДМК Пресс, 2022. – 384 с. – ISBN 978-5-97060-948-8. – Текст : непосредственный.
8. FastAPI Documentation : сайт. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 24.04.2026). – Текст : электронный.
9. Pydantic Documentation : сайт. – URL: <https://docs.pydantic.dev/> (дата обращения: 24.04.2026). – Текст : электронный.
10. Uvicorn Documentation : сайт. – URL: <https://www.uvicorn.org/> (дата обращения: 24.04.2026). – Текст : электронный.
11. Starlette Documentation : сайт. – URL: <https://www.starlette.io/> (дата обращения: 24.04.2026). – Текст : электронный.
12. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – Москва : Вильямс, 2005. – 1328 с. – ISBN 978-5-8459-0788-2. – Текст : непосредственный.

13. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – 3-е изд. – Москва : Вильямс, 2003. – 1440 с. – ISBN 978-5-8459-0527-7. – Текст : непосредственный.
14. Грофф, Дж. Р. SQL: полное руководство / Дж. Р. Грофф, П. Н. Вайнберг, Э. Дж. Оппель. – 3-е изд. – Санкт-Петербург : Диалектика, 2020. – 960 с. – ISBN 978-5-907144-26-5. – Текст : непосредственный.
15. Моргунов, Е. П. PostgreSQL. Основы языка SQL / Е. П. Моргунов. – Санкт-Петербург : БХВ-Петербург, 2022. – 336 с. – ISBN 978-5-9775-3960-9. – Текст : непосредственный.
16. PostgreSQL Documentation : сайт. – URL: <https://www.postgresql.org/docs/> (дата обращения: 24.04.2026). – Текст : электронный.
17. SQLAlchemy Documentation : сайт. – URL: <https://docs.sqlalchemy.org/> (дата обращения: 24.04.2026). – Текст : электронный.
18. Alembic – Database Migrations for SQLAlchemy : сайт. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 24.04.2026). – Текст : электронный.
19. Psycopg – PostgreSQL adapter for Python : сайт. – URL: <https://www.psycopg.org/docs/> (дата обращения: 24.04.2026). – Текст : электронный.
20. python-docx Documentation : сайт. – URL: <https://python-docx.readthedocs.io/> (дата обращения: 24.04.2026). – Текст : электронный.
21. pywin32 Documentation : сайт. – URL: <https://github.com/mhammond/pywin32> (дата обращения: 24.04.2026). – Текст : электронный.
22. Стандарт открытого формата Office Open XML : сайт. – URL: <https://www.ecma-international.org/publications-and-standards/standards/ecma-376/> (дата обращения: 24.04.2026). – Текст : электронный.
23. JWT Introduction : сайт. – URL: <https://jwt.io/introduction> (дата обращения: 24.04.2026). – Текст : электронный.
24. OAuth 2.0 Authorization Framework : сайт. – URL: <https://datatracker.ietf.org/doc/html/rfc6749> (дата обращения: 24.04.2026). – Текст : электронный.

25. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. – Санкт-Петербург : Питер, 2013. – 464 с. – ISBN 978-5-496-00487-9. – Текст : непосредственный.
26. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2018. – 352 с. – ISBN 978-5-4461-0772-8. – Текст : непосредственный.
27. Рамбо, Дж. UML 2.0. Объектно-ориентированное моделирование и разработка / Дж. Рамбо, М. Блаха. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 542 с. – ISBN 978-5-4461-9428-5. – Текст : непосредственный.
28. Unified Modeling Language Specification, Version 2.5.1 : сайт. – URL: <https://www.omg.org/spec/UML/2.5.1/PDF> (дата обращения: 24.04.2026). – Текст : электронный.
29. draw.io Documentation : сайт. – URL: <https://www.drawio.com/doc/> (дата обращения: 24.04.2026). – Текст : электронный.
30. Чакон, С. Git для профессионального программиста / С. Чакон, Б. Штрауб. – Санкт-Петербург : Питер, 2016. – 496 с. – ISBN 978-5-496-01763-3. – Текст : непосредственный.
31. Git Documentation : сайт. – URL: <https://git-scm.com/doc> (дата обращения: 24.04.2026). – Текст : электронный.
32. Дэкетт, Д. HTML и CSS. Разработка и создание веб-сайтов / Д. Дэкетт. – Москва : Эксмо, 2014. – 480 с. – ISBN 978-5-699-64193-2. – Текст : непосредственный.
33. Дэкетт, Д. JavaScript и jQuery. Интерактивная web-разработка / Д. Дэкетт. – Москва : Эксмо, 2017. – 640 с. – ISBN 978-5-699-80285-2. – Текст : непосредственный.
34. Фрейн, Б. HTML5 и CSS3. Разработка современных web-сайтов / Б. Фрейн. – 2-е изд. – Санкт-Петербург : Питер, 2022. – 384 с. – ISBN 978-5-4461-0526-2. – Текст : непосредственный.
35. HTML : сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTML> (дата обращения: 24.04.2026). – Текст : электронный.

36. CSS : сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/CSS> (дата обращения: 24.04.2026). – Текст : электронный.
37. JavaScript : сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/JavaScript> (дата обращения: 24.04.2026). – Текст : электронный.
38. HTTP : сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTTP> (дата обращения: 24.04.2026). – Текст : электронный.
39. Fielding, R. HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. – RFC 9110. – 2022. – URL: <https://www.rfc-editor.org/rfc/rfc9110.html> (дата обращения: 24.04.2026). – Текст : электронный.
40. Филдинг, Р. Архитектурный стиль REST и проектирование сетевых приложений : сайт. – URL: https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (дата обращения: 24.04.2026). – Текст : электронный.
41. Оккен, Б. pytest. Быстрое тестирование приложений на Python / Б. Оккен. – Санкт-Петербург : Питер, 2020. – 320 с. – ISBN 978-5-4461-1265-4. – Текст : непосредственный.
42. PyCharm Documentation : сайт. – URL: <https://www.jetbrains.com/pycharm/documentation/> (дата обращения: 24.04.2026). – Текст : электронный.
43. Bootstrap Documentation : сайт. – URL: <https://getbootstrap.com/docs/5.3/> (дата обращения: 24.04.2026). – Текст : электронный.
44. Docker Documentation : сайт. – URL: <https://docs.docker.com/> (дата обращения: 24.04.2026). – Текст : электронный.
45. NGINX Documentation : сайт. – URL: <https://nginx.org/en/docs/> (дата обращения: 24.04.2026). – Текст : электронный.
46. ReportLab Documentation : сайт. – URL: <https://docs.reportlab.com/> (дата обращения: 24.04.2026). – Текст : Redirecting... git-scm.com электронный.
47. asyncio Documentation : сайт. – URL: <https://docs.python.org/3/library/asyncio.html> (дата обращения: 24.04.2026). – Текст : электронный.
48. Кнут, Д. Э. Искусство программирования. Том 1. Основные алгоритмы / Д. Э. Кнут. – 4-е изд. – Москва : Вильямс, 2022. – 720 с. – ISBN 978-5-8459-2075-1. – Текст : непосредственный.

49. Лопес, Ф. SQLAlchemy. Архитектура и паттерны / Ф. Лопес. – Москва : ДМК Пресс, 2020. – 384 с. – ISBN 978-5-97060-765-0. – Текст : непосредственный.
50. Любанович, Б. Простой Python. Современный стиль программирования / Б. Любанович. – Санкт-Петербург : Питер, 2020. – 480 с. – ISBN 978-5-4461-1269-2. – Текст : непосредственный.
51. Вурс, Д. Python. Карманный справочник / Д. Вурс. – Москва : Эксмо, 2022. – 320 с. – ISBN 978-5-04-164234-5. – Текст : непосредственный.
52. Python Tutorial : сайт. – URL: <https://docs.python.org/3/tutorial/> (дата обращения: 25.04.2026). – Текст : электронный.
53. Рамальо, Л. Python. К вершинам мастерства. Том 2 / Л. Рамальо. – Москва : ДМК Пресс, 2023. – 672 с. – ISBN 978-5-97060-985-3. – Текст : непосредственный.
54. Дауни, А. Б. Think Python. Аналитический подход к программированию / А. Б. Дауни. – 3-е изд. – Санкт-Петербург : Питер, 2024. – 320 с. – ISBN 978-5-4461-2154-4. – Текст : непосредственный.